

DEPARTMENT OF INFORMATION TECHNOLOGY 24ITT32 -

PROGRAMMING IN JAVA

INDIVIDUAL PROJECT ON ACADEMIC MANAGEMENT SYSTEM

PROJECT REPORT

Submitted by

SIRIN FATHIMA M I

25ITR136

1. AIM

To develop a Java-based KEC Academic System using JDBC and MySQL to manage student academic information, attendance, assessment marks, and academic reports.

2. OBJECTIVE

To design and develop a database-driven Java application that supports student and course management, course enrollment, attendance tracking, CIA and ESE mark entry, semester mark sheets with SGPA, attendance defaulter reports, and department result summaries.

3. SOFTWARE REQUIREMENTS

- Java Development Kit (JDK) 17 or later
- Apache Maven
- MySQL Server 8.x
- MySQL Connector/J
- Command Prompt or Visual Studio Code

4. MODULES

Student Management: Adds student records with roll number, name, department, and current semester.

Course Management: Adds course code, title, credits, and course type.

Enrollment and Attendance: Enrolls a student in a course and records total classes, attended classes, and condonation status.

Assessment Marks: Enters or updates CIA-1, CIA-2, assignment/quiz/lab, and ESE marks.

Mark Sheet and SGPA: Displays course-level marks, attendance, grade, status, SGPA, and equivalent percentage.

Attendance Defaulter Report: Lists enrolled students whose attendance is below 65%.

Department Result Summary: Summarizes pass/fail counts and average SGPA by department.

5. PROCEDURE

1. Create the MySQL database and tables by executing the supplied kec_schema.sql script.
2. Configure the MySQL connection details in DBConnection.java.
3. Open Command Prompt and change to the project folder containing pom.xml.
4. Run Maven to clean and compile the project and execute MainApp.
5. Choose a menu option and enter the requested student, course, attendance, or marks data.
6. View the generated academic report and capture the actual console output for documentation.

6. FLOW OF EXECUTION

START
↓
Launch Application
↓
Connect to MySQL using JDBC
↓
Display Main Menu
↓
Select Module
↓
Read and Validate User Input
↓
Execute Database Operations
↓
Display Result
↓
Return to Main Menu
↓
Close Connection and EXIT

7.DATABASE SCHEMA

Database Name: kec_academic

Table 1: students

```
mysql> DESCRIBE students;
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| roll_no          | varchar(20)  | NO   | PRI | NULL    |       |
| student_name     | varchar(100) | NO   |     | NULL    |       |
| department       | varchar(30)  | NO   |     | NULL    |       |
| current_semester | int          | NO   |     | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
4 rows in set (0.01 sec)
```

Table 2: courses

```
mysql> DESCRIBE courses;
+-------------+-----------------------------------------+------+-----+---------+-------+
| Field       | Type                                    | Null | Key | Default | Extra |
+-------------+-----------------------------------------+------+-----+---------+-------+
| course_code | varchar(20)                             | NO   | PRI | NULL    |       |
| title       | varchar(120)                            | NO   |     | NULL    |       |
| credits     | decimal(4,1)                            | NO   |     | NULL    |       |
| course_type | enum('THEORY','PRACTICAL','EMBEDDED') | NO   |     | NULL    |       |
+-------------+-----------------------------------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Table 3: enrollments

```
mysql> DESCRIBE enrollments;
+---------------------+-------------+------+-----+---------+----------------+
| Field               | Type        | Null | Key | Default | Extra          |
+---------------------+-------------+------+-----+---------+----------------+
| enrollment_id       | int         | NO   | PRI | NULL    | auto_increment |
| roll_no             | varchar(20) | NO   | MUL | NULL    |                |
| course_code         | varchar(20) | NO   | MUL | NULL    |                |
| semester            | int         | NO   |     | NULL    |                |
| total_classes       | int         | NO   |     | 0       |                |
| classes_attended    | int         | NO   |     | 0       |                |
| medical_condonation | tinyint(1)  | NO   |     | 0       |                |
+---------------------+-------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

Table 4: assessment_marks

```
mysql> DESCRIBE assessment_marks;
+-----------------+--------------+------+-----+-------------------+-----------------------------------------------+
| Field           | Type         | Null | Key | Default           | Extra                                         |
+-----------------+--------------+------+-----+-------------------+-----------------------------------------------+
| enrollment_id   | int          | NO   | PRI | NULL              |                                               |
| cia1            | decimal(5,2) | YES  |     | NULL              |                                               |
| cia2            | decimal(5,2) | YES  |     | NULL              |                                               |
| assignment_mark | decimal(5,2) | YES  |     | NULL              |                                               |
| ese_mark        | decimal(5,2) | YES  |     | NULL              |                                               |
| ese_absent      | tinyint(1)   | NO   |     | 0                 |                                               |
| updated_at      | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+-----------------+--------------+------+-----+-------------------+-----------------------------------------------+
7 rows in set (0.00 sec)
```

8.SOURCE CODE

MainApp.java

```
package com.kec.academic;

import java.sql.SQLException;
import java.util.Scanner;

public class MainApp {
    private static final Scanner in = new Scanner(System.in);
    private static final StudentDAO students = new StudentDAO();
    private static final AcademicDAO academic = new AcademicDAO();

    public static void main(String[] args) {
        System.out.println("=====================================================");
        System.out.println(" KONGU ENGINEERING COLLEGE - R2024 ACADEMIC SYSTEM");
        System.out.println("=====================================================");
        while (true) {
            System.out.println("\n1. Add Student\n2. Add Course\n3. Enroll Student + Attendance\n4.
Enter/Update CIA and ESE Marks\n5. Update Attendance / Condonation\n6. Student Semester Mark
Sheet + SGPA\n7. Attendance Defaulter Report\n8. Department Result Summary\n0. Exit");
            int choice = number("Choose option: ");
            try {
                switch (choice) {
                    case 1 -> addStudent(); case 2 -> addCourse(); case 3 -> enroll();
                    case 4 -> marks(); case 5 -> attendance(); case 6 -> academic.printMarkSheet(read("Roll
number: "), number("Semester: "));
                    case 7 -> academic.printAttendanceDefaulters(number("Semester: "));
                    case 8 -> academic.printClassSummary(number("Semester: "));
                    case 0 -> { System.out.println("Application closed."); return; }
                    default -> System.out.println("Invalid menu option.");
                }
            } catch (SQLException | IllegalArgumentException ex) {
                System.out.println("ERROR: " + ex.getMessage());
            }
        }
    }
    private static void addStudent() throws SQLException {
        String roll=read("Roll number (e.g. 24ITR001): ").toUpperCase();
        String name=read("Student name: "); String dept=read("Department (IT/CSE/ECE/MECH):
").toUpperCase();
        int sem=number("Current semester (1-8): ");
students.addStudent(roll,name,dept,sem);System.out.println("Student added successfully.");
```

```
    }
    private static void addCourse() throws SQLException {
        String code=read("Course code (e.g. 24ITR31): ").toUpperCase();String title=read("Course title:
");double credits=decimal("Credits: ");
        String type=read("Course type (THEORY/PRACTICAL/EMBEDDED):
").toUpperCase();students.addCourse(code,title,credits,type);System.out.println("Course added
successfully.");
    }
    private static void enroll() throws SQLException {
        String roll=read("Roll number: ").toUpperCase(),course=read("Course code: ").toUpperCase();int
sem=number("Semester: ");
        int total=number("Total classes held: "),att=number("Classes attended: ");boolean
cond=yes("Medical condonation approved? (y/n): ");
        academic.enroll(roll,course,sem,total,att,cond);System.out.println("Enrollment and initial
attendance saved.");
    }
    private static void marks() throws SQLException {
        String roll=read("Roll number: ").toUpperCase(),course=read("Course code: ").toUpperCase();int
sem=number("Semester: ");
        System.out.println("Leave a mark blank to keep its existing value. Enter AB for absent ESE.");
        Double c1=optionalMark("CIA Test 1 (0-100): "),c2=optionalMark("CIA Test 2 (0-100):
"),ass=optionalMark("Assignment/Quiz/Lab mark (0-100): ");
        String eseText=read("ESE mark (0-100), AB, or blank to keep: ");boolean
absent=eseText.equalsIgnoreCase("AB");Double
ese=eseText.isBlank()||absent?null:parseMark(eseText);
        academic.updateMarks(roll,course,sem,c1,c2,ass,ese,absent);System.out.println("Assessment
marks updated successfully.");
    }
    private static void attendance() throws SQLException {
        String roll=read("Roll number: ").toUpperCase(),course=read("Course code: ").toUpperCase();int
sem=number("Semester: ");int total=number("Total classes held: "),att=number("Classes attended:
");boolean cond=yes("Medical condonation approved? (y/n): ");
        academic.updateAttendance(roll,course,sem,total,att,cond);System.out.println("Attendance updated
successfully.");
    }
    private static Double optionalMark(String prompt){String s=read(prompt);return
s.isBlank()?null:parseMark(s);}
    private static Double parseMark(String s){double d=Double.parseDouble(s);if(d<0||d>100)throw new
IllegalArgumentException("Marks must be between 0 and 100.");return d;}
    private static String read(String p){System.out.print(p);return in.nextLine().trim();}
    private static int number(String p){return Integer.parseInt(read(p));}
    private static double decimal(String p){return Double.parseDouble(read(p));}
    private static boolean yes(String p){return read(p).equalsIgnoreCase("y");}
}
```

StudentDAO.java

```
package com.kec.academic;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class StudentDAO {
    public void addStudent(String roll, String name, String dept, int semester) throws SQLException {
        String sql = "INSERT INTO students(roll_no,student_name,department,current_semester)
VALUES(?,?,?,?)";
        try (Connection con = DBConnection.getConnection(); PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setString(1, roll); ps.setString(2, name); ps.setString(3, dept); ps.setInt(4, semester);
            ps.executeUpdate();
        }
    }

    public void addCourse(String code, String title, double credits, String type) throws SQLException {
        String sql = "INSERT INTO courses(course_code,title,credits,course_type) VALUES(?,?,?,?)";
        try (Connection con = DBConnection.getConnection(); PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setString(1, code); ps.setString(2, title); ps.setDouble(3, credits); ps.setString(4,
type.toUpperCase());
            ps.executeUpdate();
        }
    }
}
```

AcademicDAO.java

```
package com.kec.academic;

import java.sql.*;

public class AcademicDAO {
    public void enroll(String roll, String course, int semester, int total, int attended, boolean condonation)
throws SQLException {
        if (total < 0 || attended < 0 || attended > total) throw new IllegalArgumentException("Attendance
values are invalid.");
```

```
        String sql = "INSERT INTO
enrollments(roll_no,course_code,semester,total_classes,classes_attended,medical_condonation)
VALUES(?,?,?,?,?,?)";
        try (Connection c=DBConnection.getConnection(); PreparedStatement p=c.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            p.setString(1,roll); p.setString(2,course); p.setInt(3,semester); p.setInt(4,total);
p.setInt(5,attended); p.setBoolean(6,condonation); p.executeUpdate();
            try(ResultSet r=p.getGeneratedKeys()) { if(r.next()) ensureMarksRow(c,r.getInt(1)); }
        }
    }
    private void ensureMarksRow(Connection c, int id) throws SQLException {
        try(PreparedStatement p=c.prepareStatement("INSERT INTO assessment_marks(enrollment_id)
VALUES(?)")) { p.setInt(1,id); p.executeUpdate(); }
    }
    private int enrollmentId(String roll, String course, int sem) throws SQLException {
        String sql="SELECT enrollment_id FROM enrollments WHERE roll_no=? AND course_code=?
AND semester=?";
        try(Connection c=DBConnection.getConnection(); PreparedStatement p=c.prepareStatement(sql))
{
            p.setString(1,roll); p.setString(2,course); p.setInt(3,sem);
            try(ResultSet r=p.executeQuery()){if(r.next())return r.getInt(1);}
        }
        throw new SQLException("Enrollment not found. Enroll the student in this course first.");
    }
    public void updateMarks(String roll,String course,int sem,Double cia1,Double cia2,Double
assignment,Double ese,boolean absent) throws SQLException {
        if(absent && ese != null) throw new IllegalArgumentException("For AB, leave ESE mark
empty.");
        validateMark(cia1); validateMark(cia2); validateMark(assignment); validateMark(ese);
        int id=enrollmentId(roll,course,sem);
        String sql="UPDATE assessment_marks SET
cia1=COALESCE(?,cia1),cia2=COALESCE(?,cia2),assignment_mark=COALESCE(?,assignment_mar
k),ese_mark=?,ese_absent=? WHERE enrollment_id=?";
        try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(sql)) {
            setNullable(p,1,cia1); setNullable(p,2,cia2); setNullable(p,3,assignment);
setNullable(p,4,absent?null:ese); p.setBoolean(5,absent); p.setInt(6,id); p.executeUpdate();
        }
    }
    private void validateMark(Double x) { if(x!=null && (x<0 || x>100)) throw new
IllegalArgumentException("Marks must be between 0 and 100."); }
    private void setNullable(PreparedStatement p,int index,Double x)throws SQLException {
if(x==null)p.setNull(index,Types.DECIMAL);else p.setDouble(index,x); }
```

```
    public void updateAttendance(String roll,String course,int sem,int total,int attended,boolean
condonation)throws SQLException {
        if(total<0||attended<0||attended>total)throw new IllegalArgumentException("Attendance values are
invalid.");
        String sql="UPDATE enrollments SET total_classes=?,classes_attended=?,medical_condonation=?
WHERE roll_no=? AND course_code=? AND semester=?";
        try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(sql))
{p.setInt(1,total);p.setInt(2,attended);p.setBoolean(3,condonation);p.setString(4,roll);p.setString(5,cours
e);p.setInt(6,sem);if(p.executeUpdate()==0)throw new SQLException("Enrollment not found.");}
    }

    private String grade(double total) { if(total>=91)return "O";if(total>=81)return
"A+";if(total>=71)return "A";if(total>=61)return "B+";if(total>=50)return "B";return "RA"; }
    private int points(String g) { return switch(g){case "O"->10;case "A+"->9;case "A"->8;case "B+"-
>7;case "B"->6;default->0;}; }

    public void printMarkSheet(String roll,int sem)throws SQLException {
        String sql="SELECT
s.roll_no,s.student_name,s.department,e.semester,e.total_classes,e.classes_attended,e.medical_condonati
on,c.course_code,c.title,c.credits,c.course_type,m.cia1,m.cia2,m.assignment_mark,m.ese_mark,m.ese_a
bsent FROM enrollments e JOIN students s ON s.roll_no=e.roll_no JOIN courses c ON
c.course_code=e.course_code JOIN assessment_marks m ON m.enrollment_id=e.enrollment_id
WHERE s.roll_no=? AND e.semester=? ORDER BY c.course_code";
        double weightedPoints=0,credits=0;boolean found=false;
        System.out.println("\n================ KEC R2024 ACADEMIC MARK SHEET
================");
        System.out.printf("%-10s %-22s %-8s %-8s %-8s %-8s %-6s %-5s %-
5s%n","COURSE","TITLE","CIA/100","ESE/100","TOTAL","ATTEND%","GRADE","GP","STATU
S");
        try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(sql)) {
            p.setString(1,roll);p.setInt(2,sem);
            try(ResultSet r=p.executeQuery()) { while(r.next()) { found=true;
                String type=r.getString("course_type");double credit=r.getDouble("credits");int
totalClasses=r.getInt("total_classes"),attended=r.getInt("classes_attended");
                double attendance=totalClasses==0?0:100.0*attended/totalClasses;
                Double
a=nullable(r,"cia1"),b=nullable(r,"cia2"),d=nullable(r,"assignment_mark"),ese=nullable(r,"ese_mark");
                boolean absent=r.getBoolean("ese_absent"),cond=r.getBoolean("medical_condonation");
                double ciaAverage=average(a,b,d); double
ciaWeight=type.equals("THEORY")?.40:type.equals("PRACTICAL")?.60:.50;
                double eseWeight=1-ciaWeight; double
eseThreshold=type.equals("THEORY")?27:type.equals("PRACTICAL")?20:22.5;
                double ciaComponent=ciaAverage*ciaWeight; double
eseComponent=ese==null?0:ese*eseWeight;double total=ciaComponent+eseComponent;
```

```
            String g,status;boolean eligible=attendance>=75||(attendance>=65&&cond);
            if(attendance<65){g="SA";status="BARRED";}else
if(attendance<75&&!cond){g="SA";status="CONDONATION NEEDED";}else
if(absent){g="AB";status="FAIL";}else if(ese==null){g="PENDING";status="ESE PENDING";}else
if(eseComponent<eseThreshold||total<50){g="RA";status="FAIL";}else{g=grade(total);status="PASS";
}
            int gp=points(g);if(!g.equals("PENDING")){weightedPoints+=credit*gp;credits+=credit;}
            String ciaText=(a==null&&b==null&&d==null)?"-":String.format("%.1f",ciaAverage);
            String eseText=absent?"AB":ese==null?"-":String.format("%.1f",ese);
            String totalText=g.equals("PENDING")?"-":String.format("%.1f",total);
            System.out.printf("%-10s %-22.22s %-8s %-8s %-8s %-8.1f %-6s %-5d %-
5s%n",r.getString("course_code"),r.getString("title"),ciaText,eseText,totalText,attendance,g,gp,status);
          }}
        }
        if(!found){System.out.println("No enrollments found for this roll number and semester.");return;}
        double sgpa=credits==0?0:weightedPoints/credits;
        System.out.printf("SGPA: %.2f | Equivalent Percentage: %.2f%%%n",sgpa,sgpa*10);
        System.out.println("Note: CIA component = average of CIA1, CIA2 and assignment mark × CIA
weight; missing CIA entries are treated as zero until entered.");

System.out.println("============================================================
===\n");
    }
    private Double nullable(ResultSet r,String col)throws SQLException {double
x=r.getDouble(col);return r.wasNull()?null:x;}
    private double average(Double... values){double sum=0;for(Double
v:values)sum+=(v==null?0:v);return sum/values.length;}

    public void printAttendanceDefaulters(int sem)throws SQLException {
        String sql="SELECT
s.roll_no,s.student_name,s.department,e.course_code,e.total_classes,e.classes_attended FROM
enrollments e JOIN students s ON s.roll_no=e.roll_no WHERE e.semester=? AND e.total_classes>0
AND 100.0*e.classes_attended/e.total_classes<65 ORDER BY s.department,s.roll_no";
        try(Connection c=DBConnection.getConnection();PreparedStatement
p=c.prepareStatement(sql)){p.setInt(1,sem);try(ResultSet r=p.executeQuery()){System.out.println("\n---
ATTENDANCE DEFAULTERS (SA: below 65%) ---");while(r.next())System.out.printf("%s | %-22s |
%s | %s | %.1f%% | Grade:
SA%n",r.getString("roll_no"),r.getString("student_name"),r.getString("department"),r.getString("course
_code"),100.0*r.getInt("classes_attended")/r.getInt("total_classes"));}}
    }
    public void printClassSummary(int sem)throws SQLException {
        String sql="SELECT
s.department,s.roll_no,e.total_classes,e.classes_attended,e.medical_condonation,c.credits,c.course_type,
m.cia1,m.cia2,m.assignment_mark,m.ese_mark,m.ese_absent FROM enrollments e JOIN students s ON
```

```
s.roll_no=e.roll_no JOIN courses c ON c.course_code=e.course_code JOIN assessment_marks m ON
m.enrollment_id=e.enrollment_id WHERE e.semester=?";
        class Row { double pts=0,cr=0; int pass=0,fail=0; }
        java.util.Map<String,Row> map=new java.util.TreeMap<>();
        try(Connection c=DBConnection.getConnection();PreparedStatement
p=c.prepareStatement(sql)){p.setInt(1,sem);try(ResultSet r=p.executeQuery()){while(r.next()){
            String dept=r.getString("department"),type=r.getString("course_type");Row
row=map.computeIfAbsent(dept,k->new Row());
            int t=r.getInt("total_classes"),a=r.getInt("classes_attended");double
att=t==0?0:100.0*a/t;boolean
cond=r.getBoolean("medical_condonation"),abs=r.getBoolean("ese_absent");Double
x=nullable(r,"cia1"),y=nullable(r,"cia2"),z=nullable(r,"assignment_mark"),ese=nullable(r,"ese_mark");
            if(att<65||(att<75&&!cond)||abs||ese==null) {
if(ese!=null||abs||att<65||(att<75&&!cond))row.fail++; continue; }
            double cw=type.equals("THEORY")?.4:type.equals("PRACTICAL")?.6:.5;double
total=average(x,y,z)*cw+ese*(1-cw);double
threshold=type.equals("THEORY")?27:type.equals("PRACTICAL")?20:22.5;
            String g=(total>=50&&ese*(1-cw)>=threshold)?grade(total):"RA";int
gp=points(g);row.pts+=r.getDouble("credits")*gp;row.cr+=r.getDouble("credits");if(gp>0)row.pass++;e
lse row.fail++;
        }}}
        System.out.println("\n--- DEPARTMENT RESULT SUMMARY ---");System.out.printf("%-12s
%-10s %-10s %-12s%n","DEPARTMENT","PASS","FAIL/SA","AVG SGPA");
        for(var e:map.entrySet()){Row r=e.getValue();System.out.printf("%-12s %-10d %-10d %-
12.2f%n",e.getKey(),r.pass,r.fail,r.cr==0?0:r.pts/r.cr);}
    }
}
```

DBConnection.java

```
package com.kec.academic;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class DBConnection {
    private static final String URL =
"jdbc:mysql://localhost:3306/kec_academic?serverTimezone=Asia/Kolkata";
    private static final String USER = "root";
    // Change this to your own MySQL password.
    private static final String PASSWORD = "YOUR_MYSQL_PASSWORD";
```

```
    private DBConnection() { }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

kec_schema.sql

```
CREATE DATABASE IF NOT EXISTS kec_academic;
USE kec_academic;

CREATE TABLE IF NOT EXISTS students (
  roll_no VARCHAR(20) PRIMARY KEY,
  student_name VARCHAR(100) NOT NULL,
  department VARCHAR(30) NOT NULL,
  current_semester INT NOT NULL CHECK (current_semester BETWEEN 1 AND 8)
);
CREATE TABLE IF NOT EXISTS courses (
  course_code VARCHAR(20) PRIMARY KEY,
  title VARCHAR(120) NOT NULL,
  credits DECIMAL(4,1) NOT NULL CHECK (credits > 0),
  course_type ENUM('THEORY','PRACTICAL','EMBEDDED') NOT NULL
);
CREATE TABLE IF NOT EXISTS enrollments (
  enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
  roll_no VARCHAR(20) NOT NULL,
  course_code VARCHAR(20) NOT NULL,
  semester INT NOT NULL CHECK (semester BETWEEN 1 AND 8),
  total_classes INT NOT NULL DEFAULT 0 CHECK (total_classes >= 0),
  classes_attended INT NOT NULL DEFAULT 0 CHECK (classes_attended >= 0),
  medical_condonation BOOLEAN NOT NULL DEFAULT FALSE,
  UNIQUE (roll_no, course_code, semester),
  CHECK (classes_attended <= total_classes),
  FOREIGN KEY (roll_no) REFERENCES students(roll_no) ON DELETE CASCADE,
  FOREIGN KEY (course_code) REFERENCES courses(course_code) ON DELETE CASCADE
);
CREATE TABLE IF NOT EXISTS assessment_marks (
  enrollment_id INT PRIMARY KEY,
  cia1 DECIMAL(5,2) NULL CHECK (cia1 BETWEEN 0 AND 100),
  cia2 DECIMAL(5,2) NULL CHECK (cia2 BETWEEN 0 AND 100),
  assignment_mark DECIMAL(5,2) NULL CHECK (assignment_mark BETWEEN 0 AND 100),
  ese_mark DECIMAL(5,2) NULL CHECK (ese_mark BETWEEN 0 AND 100),
  ese_absent BOOLEAN NOT NULL DEFAULT FALSE,
```

```
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  FOREIGN KEY (enrollment_id) REFERENCES enrollments(enrollment_id) ON DELETE
CASCADE,
  CHECK (ese_absent = FALSE OR ese_mark IS NULL)
);
```

9. OUTPUT SCREENSHOTS

Figure 1: Main Menu. The screenshot supplied by the student confirms that Maven compiled the project and the application displayed the menu.

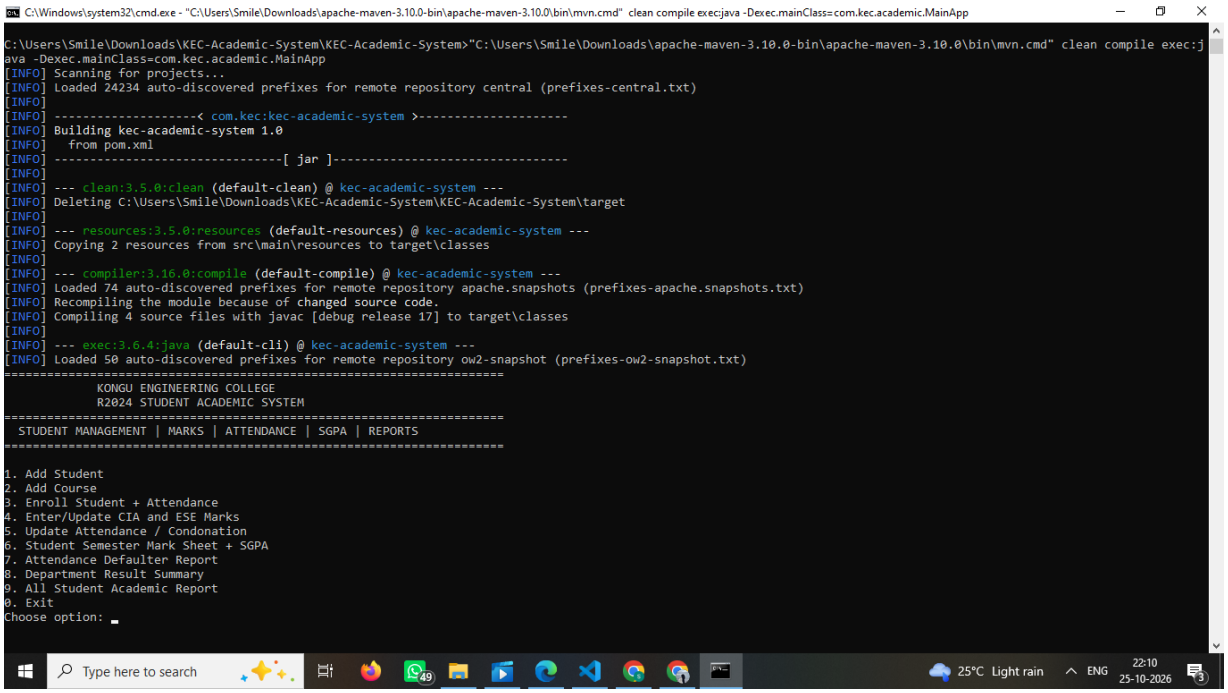


Figure 1: Main Menu of KEC Academic System

Figure 2: Add Student

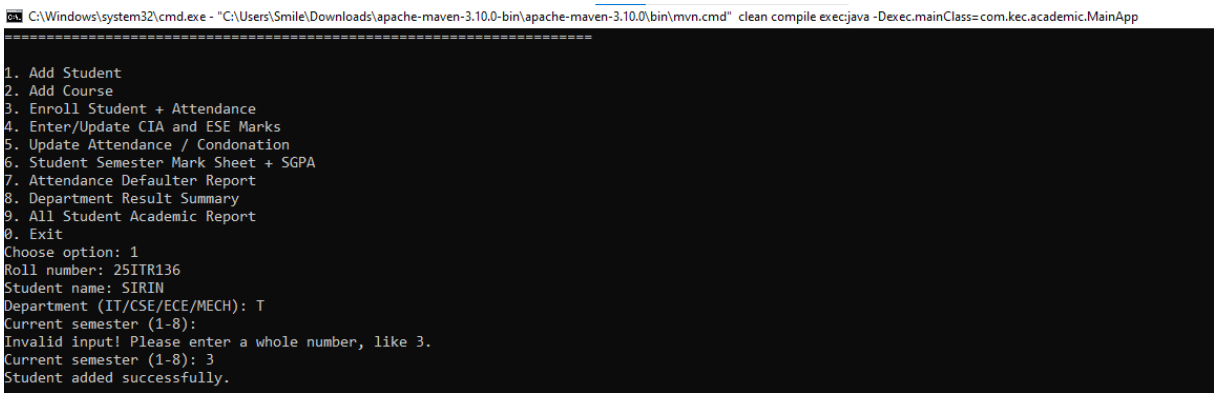


Figure 3: Add Course



Figure 4: Enroll Student and Attendance



Figure 5: Enter/Update CIA and ESE Marks



Figure 6: Update Attendance / Condonation



Figure 7: Student Semester Mark Sheet and SGPA

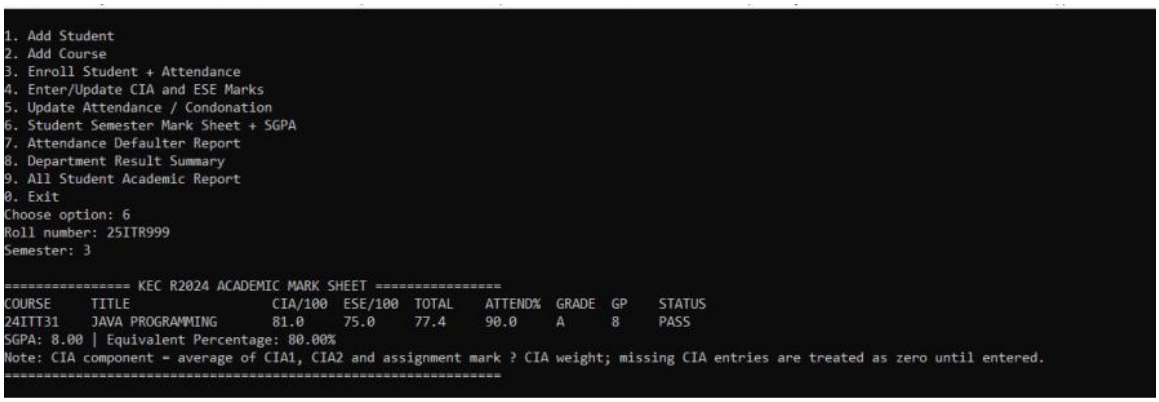


Figure 8: Attendance Defaulter Report

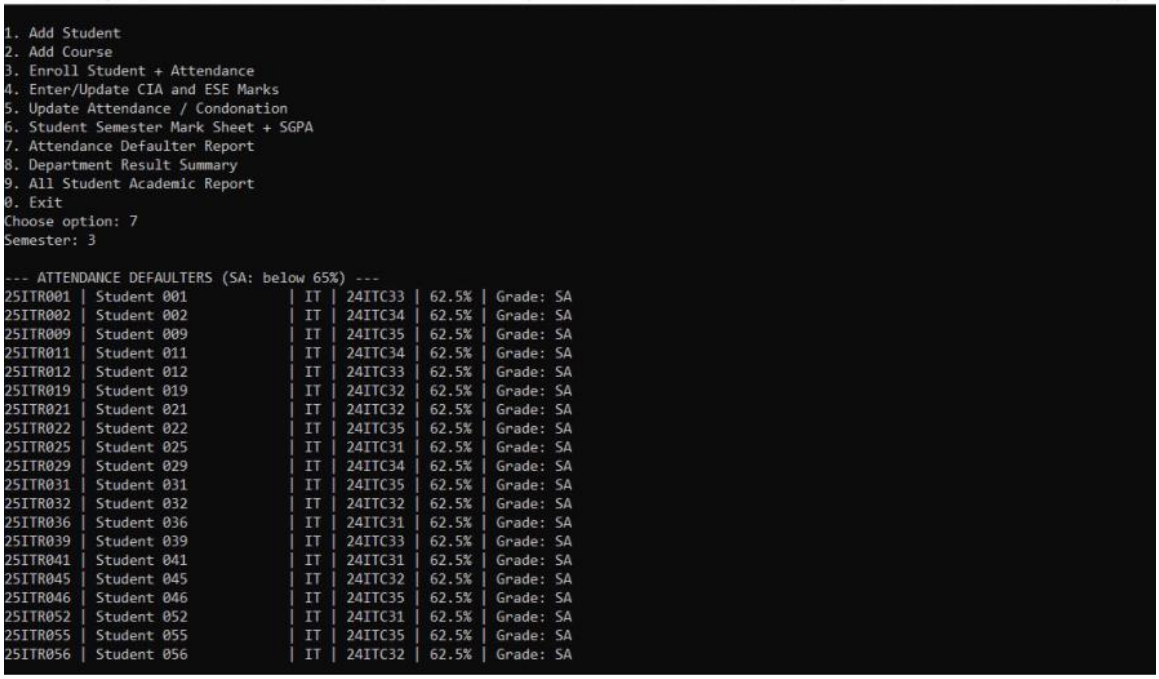
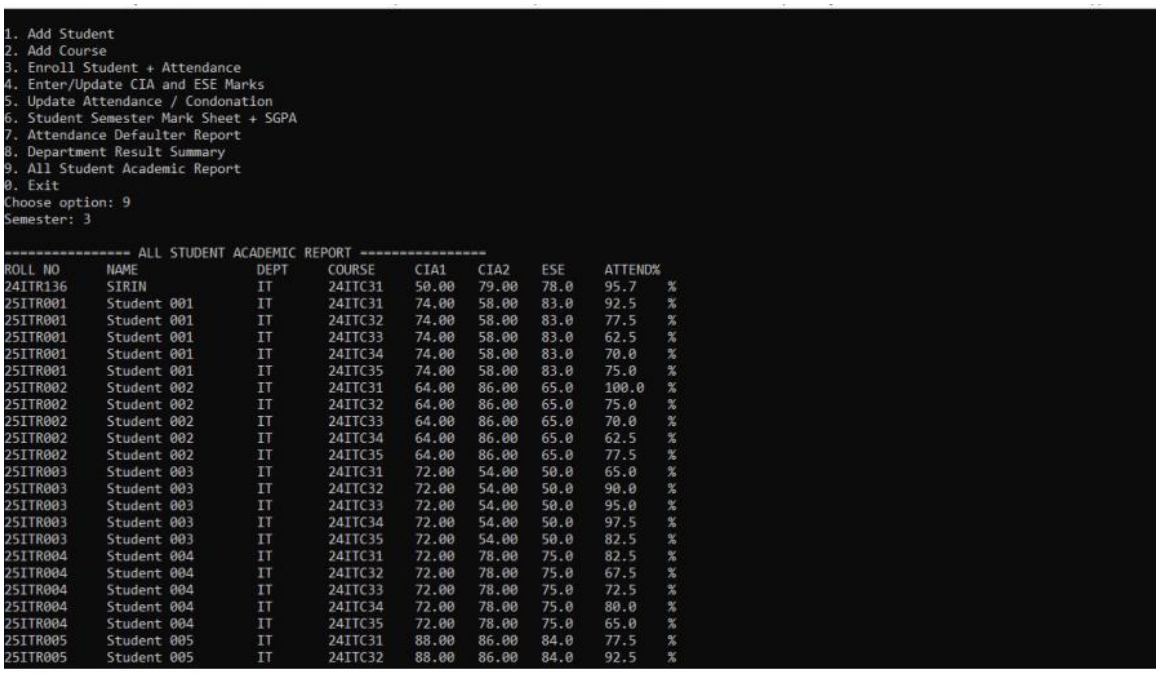


Figure 9: Department Result Summary



Figure 10: All Student Academic Report



```
1. Add Student
2. Add Course
3. Enroll Student + Attendance
4. Enter/Update CIA and ESE Marks
5. Update Attendance / Condonation
6. Student Semester Mark Sheet + SGPA
7. Attendance Defaulter Report
8. Department Result Summary
9. All Student Academic Report
0. Exit
Choose option: 9
Semester: 3

================ ALL STUDENT ACADEMIC REPORT ================
ROLL NO     NAME            DEPT    COURSE    CIA1   CIA2   ESE    ATTEND%
24ITR136    SIRIN           IT      24ITC31   50.00  79.00  78.0   95.7   %
25ITR001    Student 001     IT      24ITC31   74.00  58.00  83.0   92.5   %
25ITR001    Student 001     IT      24ITC32   74.00  58.00  83.0   77.5   %
25ITR001    Student 001     IT      24ITC33   74.00  58.00  83.0   62.5   %
25ITR001    Student 001     IT      24ITC34   74.00  58.00  83.0   70.0   %
25ITR001    Student 001     IT      24ITC35   74.00  58.00  83.0   75.0   %
25ITR002    Student 002     IT      24ITC31   64.00  86.00  65.0   100.0  %
25ITR002    Student 002     IT      24ITC32   64.00  86.00  65.0   75.0   %
25ITR002    Student 002     IT      24ITC33   64.00  86.00  65.0   70.0   %
25ITR002    Student 002     IT      24ITC34   64.00  86.00  65.0   62.5   %
25ITR002    Student 002     IT      24ITC35   64.00  86.00  65.0   77.5   %
25ITR003    Student 003     IT      24ITC31   72.00  54.00  50.0   65.0   %
25ITR003    Student 003     IT      24ITC32   72.00  54.00  50.0   90.0   %
25ITR003    Student 003     IT      24ITC33   72.00  54.00  50.0   95.0   %
25ITR003    Student 003     IT      24ITC34   72.00  54.00  50.0   97.5   %
25ITR003    Student 003     IT      24ITC35   72.00  54.00  50.0   82.5   %
25ITR004    Student 004     IT      24ITC31   72.00  78.00  75.0   82.5   %
25ITR004    Student 004     IT      24ITC32   72.00  78.00  75.0   67.5   %
25ITR004    Student 004     IT      24ITC33   72.00  78.00  75.0   72.5   %
25ITR004    Student 004     IT      24ITC34   72.00  78.00  75.0   80.0   %
25ITR004    Student 004     IT      24ITC35   72.00  78.00  75.0   65.0   %
25ITR005    Student 005     IT      24ITC31   88.00  86.00  84.0   77.5   %
25ITR005    Student 005     IT      24ITC32   88.00  86.00  84.0   92.5   %
```

10. RESULT

The KEC Academic System project was compiled and launched using Maven, and the main menu was displayed successfully in Command Prompt. The application includes student/course management, enrollment and attendance, assessment marks, mark sheet and SGPA, attendance defaulter, and department summary modules. The remaining feature outputs should be captured from the running application and inserted above before final submission.

11. GITHUB REPOSITORY

Repository URL: https://github.com/Sirin2008/KEC-Academic-System

12. REFERENCES

7. Oracle. Java Documentation. https://docs.oracle.com/en/java/
8. MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
9. MySQL. Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/